

T2.1.D.1 · Write-ahead reservation + reconciliación post-crash

Owner: Celestino · 2026-05-06 · branch *pr/t2.1.d-credit-reservations-audit-pruning* · fix mínimo sobre T2.1.D · sin deploy · sin migraciones en prod · sin borrado de datos.

Mitigación del riesgo residual del PR T2.1.D: el *crash* entre *billing.cobrar()* y el *INSERT* en *automation_reservas_credito* ya no puede dejar saldo descontado sin reserva visible. Se invirtió el orden (la fila ahora se persiste como *'preparing'* ANTES del cobro) y se sumó una función de reconciliación que repara cualquier fila *preparing* orfana en base a la huella *TransaccionCredito.job_id=accion_id*. Cubierto por 8 tests nuevos.

1. Origen del riesgo y opciones evaluadas

El PR T2.1.D dejó documentado un riesgo residual catalogado como **severidad Baja**: si la app crashea entre *billing.cobrar()* (sesión A, ya commitada) y *_persistir_reserva_nueva()* (sesión B), el saldo queda descontado y la *TransaccionCredito* persistida, pero la fila en *automation_reservas_credito* no existe.

El usuario pidió formalmente resolverlo y propuso tres opciones:

Opción	Idea	Decisión
1	Crear reserva en <i>'preparing'</i> ANTES de cobrar y completarla a <i>'pending'</i> si el pago es del cobro.	Elegida · como safety net del happy path.
2	Reconciliar a posteriori detectando <i>TransaccionCredito</i> sin reserva asociada.	Elegida · como safety net del happy path.
3	Demostrar que cobrar + crear reserva es una transacción atómica.	Descartada : <i>cobrar()</i> abre su propia sesión.

2. Diseño del fix

2.1 Nuevo flujo de reservar()

```
async def reservar(accion_id, telefono, creditos, razon):
    # 1. Lookup idempotente · si existe pending/confirmed
    #   devolver tal cual. Si existe 'preparing' → reconciliar
    #   inline antes de decidir (auto-reparación del reintento
    #   del worker).
    existing = await obtener_reserva(accion_id)
    if existing and existing['estado'] == 'preparing':
        existing = await reconciliar_accion(accion_id,
                                           max_edad_segundos=0)

    if existing and existing['estado'] in ('pending', 'confirmed'):
        return existing

    if creditos == 0:
        return _persistir_reserva(..., estado='pending')

    # 2. Write-ahead · fila 'preparing' antes de tocar saldo
    await _persistir_reserva(..., estado='preparing')

    # 3. Cobro real · atómico en sesión propia de billing
    try:
        await billing.cobrar(..., job_id=accion_id)
    except SaldoInsuficienteError:
        await _actualizar_estado(accion_id, 'failed')
        await registrar_evento('credits_reservation_failed', ...)
        raise CreditosInsuficientesError(...)

    # 4. Buscar la tx que acabamos de crear y completar reserva
```

```

tx_id = await _ultima_tx_id_para_accion(accion_id, ...)
final = await _actualizar_estado(accion_id, 'pending',
                                transaccion_credito_id=tx_id)
await registrar_evento('credits_reserved', ...)
return final

```

2.2 Función de reconciliación

reconciliar_accion(accion_id) repara una fila *'preparing'* que quedó huérfana tras un crash. La decisión depende de si existe una *TransaccionCredito* asociada por (*job_id=accion_id, telefono, delta=-creditos*):

Escenario detectado	Acción de reconciliación	Audit emitido
Existe tx de cobro (crash post-cobro)	Avanza fila a <code>'pending'</code> + linkea <code><transaccion_credito_id></code> a <code><credits_reservation_id></code>	<code><credits_reservation_reconciled></code> con resultado <code><credits_reservation_reconciled></code>
No existe tx y la fila tiene edad $\geq$ <code><max_edad_segundos></code> (saldo nunca se movió)	Marcas <code><failed></code> (saldo nunca se movió)	<code><credits_reservation_reconciled></code> con resultado <code><credits_reservation_reconciled></code>
No existe tx pero la fila es reciente (<code><max_edad_segundos></code> puede estar legítimamente en <code><ninguno></code> otro worker)	No acción	<code><credits_reservation_reconciled></code> con resultado <code><credits_reservation_reconciled></code>

reconciliar_reservas() hace el mismo trabajo en batch · soporta *dry_run* para previsualizar conteos sin tocar estado ni emitir audit. *limit* tope (1..5000) y *max_edad_segundos* default 120s.

2.3 Por qué la huella es deterministic

billing.cobrar() inserta la fila en *transacciones_credito* con *job_id, telefono* y *delta=-creditos*. T2.1.D ya pasa *job_id=accion_id*. La combinación (*job_id, telefono, delta*) es única para esta acción (y, en el caso extremo de duplicidad, la consulta toma la **más reciente por id descendente**, que es exactamente la del cobro previo al crash). El test *test_reserva_exitosa_link_transaccion_credito_id* valida el link en el happy path.

3. Archivos modificados

Archivo	Cambio
agent/automation/credits.py	Reescrito: write-ahead (<i>preparing</i> antes del cobro), <i>reconciliar_accion()</i> y <i>reconciliar_marca()</i> .
agent/automation/audit.py	<i>EVENTOS_VALIDOS</i> + <i>credits_reservation_reconciled</i> .
agent/main.py	GET <i>/admin/automation/credits/reconciliar/preview</i> · POST <i>/admin/automation/credits/reservar</i>
tests/test_automation_creditos_reservas.py	+ 8 tests: <i>TestReconciliacionCrashEntreCobroYReserva</i> (6) y <i>TestWriteAheadOrden</i> (2)

3.1 Migraciones de schema

Ninguna nueva. El estado *'preparing'* reutiliza la columna *estado VARCHAR(20)* existente. La columna *transaccion_credito_id* existente acepta el link. No hay *ALTER TABLE* ni nueva fila en *MIGRACIONES_AUTOMATION*. Compatible 100% con datos existentes.

4. Cobertura de tests del fix

Test	Caso simulado	Aserción clave
test_reconcilia_promueve_preparing_a_pending	Crash post-cobro · fila preparing + tx existente	estado → pending · transaccion_credito_id linkado
test_reconcilia_marca_failed_si_no_hay_tx_y_cobra	Crash pre-cobro · fila preparing vieja · sin tx	estado → failed · saldo intacto
test_reconcilia_no_toca_preparing_reciente	Worker en vuelo · fila preparing joven	estado → preparing (no se toca)
test_reservar_post_crash_auto_reconcilia	Worker reintentó ejecutar_accion tras crash	reservar() devuelve pending sin re-cobrar
test_reconciliar_emite_audit	Reconciliación exitosa	audit 'credits_reservation_reconciled' con resultado
test_reconciliar_reservas_batch_promueve_y_falla	Mezcla de 3 filas: tx+preparing, vieja sin tx, joven sin tx	preview cuenta correctamente · ejecución produce
test_si_cobro_falla_la_fila_existe_como_failed	Saldo insuficiente	Existe fila (failed), no inexistente
test_reserva_exitosa_link_transaccion_credito_id	Happy path	transaccion_credito_id apunta a la tx real con delta

5. Verificaciones · comandos y resultados

```
pytest tests/test_automation_creditos_reservas.py
→ 27 passed in 20.23s · 19 previos + 8 nuevos

pytest tests/test_automation_*.py
→ 205 passed in 110.88s · 197 previos + 8 nuevos

pytest
→ 867 passed in 224.63s · 859 previos + 8 nuevos
  0 failed · 0 regresión
```

6. Riesgo residual final

Escenario	Estado post-fix
Crash entre <i>_persistir(preparing)</i> y <i>cobrar()</i>	fila preparing sin tx · reconciliar la marca failed (saldo intacto) · reintento natural
Crash entre <i>cobrar()</i> y <i>_actualizar(pending)</i>	cubierto · fila preparing + tx · reconciliar la promueve a pending · saldo
Crash dentro de <i>cobrar()</i> (commit a medias)	Imposible · <i>cobrar()</i> usa transacción única SQL (UPDATE saldo + INSERT tx)
Doble cobro por reintento del worker	Imposible · <i>UNIQUE(accion_id)</i> + auto-reconcile inline en <i>reservar()</i> devuelve

Reconciliar promueve fila a pending pero no hay realmente tx

Imposible · la búsqueda exige tx con (job_id, telefono, delta=-creditos)

Adversario inyecta tx con job_id falso

Solo cuentas admin con acceso DB pueden insertar en transacciones_creditos

7. Operación · endpoints admin

```
# Preview (dry-run · NO toca estado)
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
  "$URL/admin/automation/credits/reconciliar/preview\
?max_edad_segundos=120&limit;=500"
→ { dry_run: true, total_preparing, promoted_estimado,
    marked_failed_estimado, intactas_estimado }

# Ejecutar reconciliación
curl -s -X POST -H "Authorization: Bearer $ADMIN_TOKEN" \
  "$URL/admin/automation/credits/reconciliar\
?max_edad_segundos=120&limit;=500"
→ { dry_run: false, total_preparing_inicial, promoted,
    marked_failed, intactas }
```

Recomendación operativa: ejecutar reconciliación periódica (cada 5-15 min) vía cron externo o vía un job de *arg* en el worker de Render. Sin embargo, el auto-reconcile inline al inicio de *reservar()* hace que la reconciliación programada sea opcional para el happy path del reintento del worker.

8. PR · cómo se anexa al PR T2.1.D

Branch: *pr/t2.1.d-credit-reservations-audit-pruning* · ya pusheada · este commit se suma como *follow-up* en la misma branch antes de mergear.

Mensaje de commit propuesto:

```
fix(automation): write-ahead reservation + reconciliacion
post-crash (T2.1.D.1)

Mitiga el riesgo residual de T2.1.D: crash entre billing.cobrar
y el INSERT de la reserva podia dejar saldo descontado sin
reserva visible.

- Estado 'preparing' como write-ahead antes del cobro
- reconciliar_accion / reconciliar_reservas (dry-run + ejecucion)
- Audit credits_reservation_reconciled
- 2 endpoints admin GET/POST /admin/automation/credits/reconciliar
- 8 tests nuevos (27 total · suite completa 867 passed · 0 fail)

Sin nuevas migraciones · usa columna estado VARCHAR(20)
existente.
```

9. Próximo paso

Con T2.1.D + T2.1.D.1 cerrados, el ciclo billing del Action Center es seguro contra reintento, crash y saldo insuficiente. El siguiente bloque natural es **T2.1.E** (scheduler periódico para detectar oportunidades) o **T2.2** (primera acción HIGH real con preview-then-commit conectado a Whapi). Recomendación: cerrar PR T2.1.D, hacer redeploy a Render y monitorear logs por ~24h antes de avanzar.